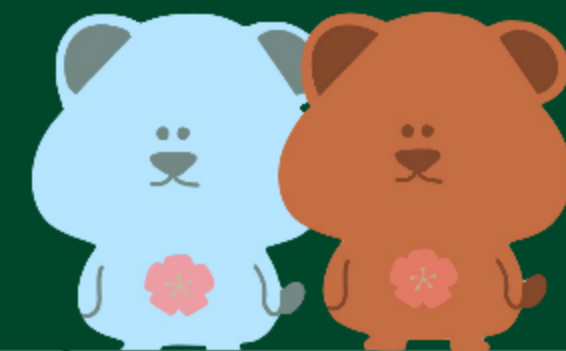




5장. 합성곱 신경망 – Part2

김서영, 박린, 안지현

목차

#01 5.3 전이 학습(1)

#02 5.3 전이 학습(2)

#03 5.4 설명 가능한 CNN
5.5 그래프 합성곱 네트워크



5.3 전이 학습(1)



5.3.1 특성 추출 기법

전이 학습(Transfer Learning): 왜, 무엇, 어떻게

문제/필요성

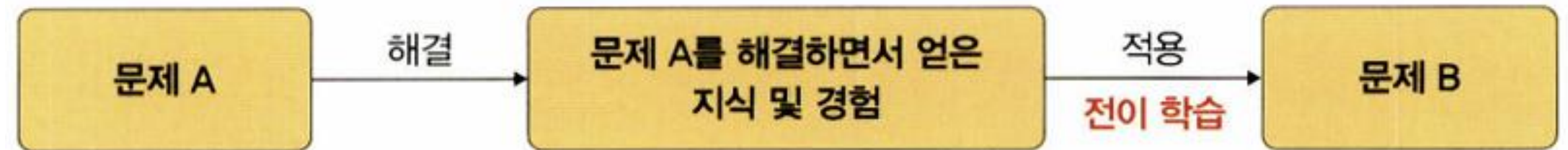
- 합성곱 신경망을 제대로 학습하려면 **큰 데이터·시간·비용**이 필요
- 현실적으로 **충분히 큰 데이터셋 확보가 어려움**
- ⇒ 이 한계를 완화하는 방법이 **전이 학습**

정의

- 정의: 대규모 데이터셋(예: ImageNet)으로 학습된 **사전 훈련 모델의 가중치**를 가져와
새 과제에 맞게 **보정(적용)**하는 것
- 사전 훈련된 모델(네트워크) = **pretrained model**

효과

- 적은 데이터로도 원하는 과제를 해결 가능
- 학습 시간 **단축**, **일반화 성능 확보**



전이 학습 방법 2가지

- ① **특성 추출(feature extraction)**: 합성곱층 고정, 분류기만 학습
- ② **미세 조정(fine-tuning)**: 상위층(또는 전층)을 작은 학습률로 재학습

5.3.1 특성 추출 기법

5.3.1 특성 추출(Feature Extraction) 기법

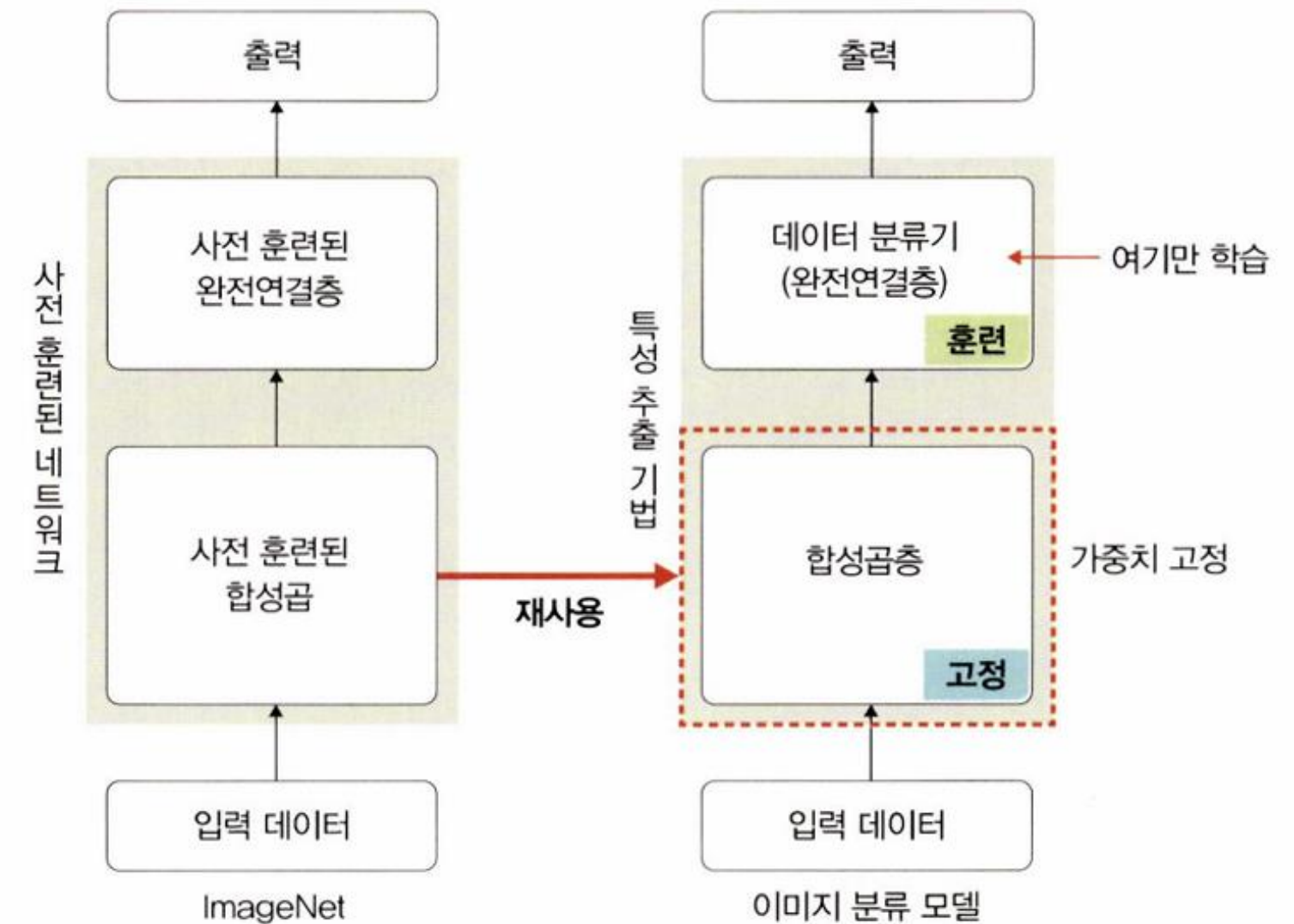
① 정의

• ImageNet 등으로 학습된 사전 훈련 모델의 가중치를 가져와 합성곱층(백본)은 고정하고 마지막 분류기(완전연결층)만 새로 학습하는 방법

② 분류 네트워크의 두 부분

• 합성곱층(+풀링): 일반적·저수준 특징 추출부 → 가중치 고정
• 데이터 분류기(완전연결층): 추출된 특징으로 최종 클래스 결정 → 여기만 학습

③ 동작 흐름도



5.3.1 특성 추출 기법

5.3.1 특성 추출(Feature Extraction) 기법

④ 기대효과

- 적은 데이터/짧은 시간으로 학습
- 사전훈련이 학습한 **보편 특징 재사용** → 일반화 도움

⑤ 사용 가능한 백본(예시)

- Xception / Inception V3 / ResNet50 / VGG16 / VGG19 / MobileNet

⑥ 주의점

- 사전훈련 시 **입력 크기·정규화(mean/std)** 가정에 맞추기
- 필요 시 다음 단계로 **미세 조정(fine-tuning)** 진행

5.3.1 특성 추출 기법

1. 라이브러리 설치와 импорт

```
# 시스템/입출력 유틸
import os, time, copy, glob, shutil
import cv2          # OpenCV (설치 필요)

# PyTorch & TorchVision
import torch
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader

# 시각화
import matplotlib.pyplot as plt
```

역할 요약

- cv2**: 이미지 조작/확인(선택적으로 활용)
- torchvision**: 모델·데이터셋·변환(Transforms)
- nn / optim**: 네트워크 층, 손실/옵티마이저
- DataLoader**: 배치 로딩·셔플·병렬화
- matplotlib**: 학습곡선·예측 결과 시각화

5.3.1 특성 추출 기법

2. 전처리 파이프라인과 데이터 로더

```
data_path = '/content/drive/MyDrive/catanddog'
```

```
transform = transforms.Compose([
    transforms.Resize([256, 256]),
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
])
```

```
train_dataset = torchvision.datasets.ImageFolder(
    data_path,
    transform=transform
)
```

```
train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=32,
    num_workers=8,
    shuffle=True
)
```

```
print(len(train_dataset))
```

```
483
```

- ① **Resize**: 입력 크기 **256×256**으로 맞춤
- ② **RandomResizedCrop(224)**: 랜덤 크롭 후 224로 리사이즈 → 데이터 확장/일반화
- ③ **RandomHorizontalFlip**: 좌우 반전으로 변이 추가
- ④ **ToTensor**: 이미지 → **Tensor** 변환
- ⑤ **batch_size=32**: 한 번에 처리할 샘플 수
- ⑥ **num_workers=8**: 병렬 로딩 프로세스 수 (과도하면 메모리 부족/오류 가능)
- ⑦ **shuffle=True**: 에폭마다 무작위 섞기

5.3.1 특성 추출 기법

*학습용 배치 24장 미리보기 (PyTorch → Matplotlib)

- PyTorch 텐서: **(B, C, H, W)** = 배치·채널·세로·가로
 - matplotlib는 **(H, W, C)**를 기대 → **축 순서 바꾸기 필요**
- `np.transpose(img, (1,2,0))` : **CHW → HWC**
- 배치 뽑기: `samples, labels = next(iter(train_loader))`
- 클래스 이름: `train_dataset.classes` (예: ['cat','dog'])

```
samples, labels = next(iter(train_loader))
```

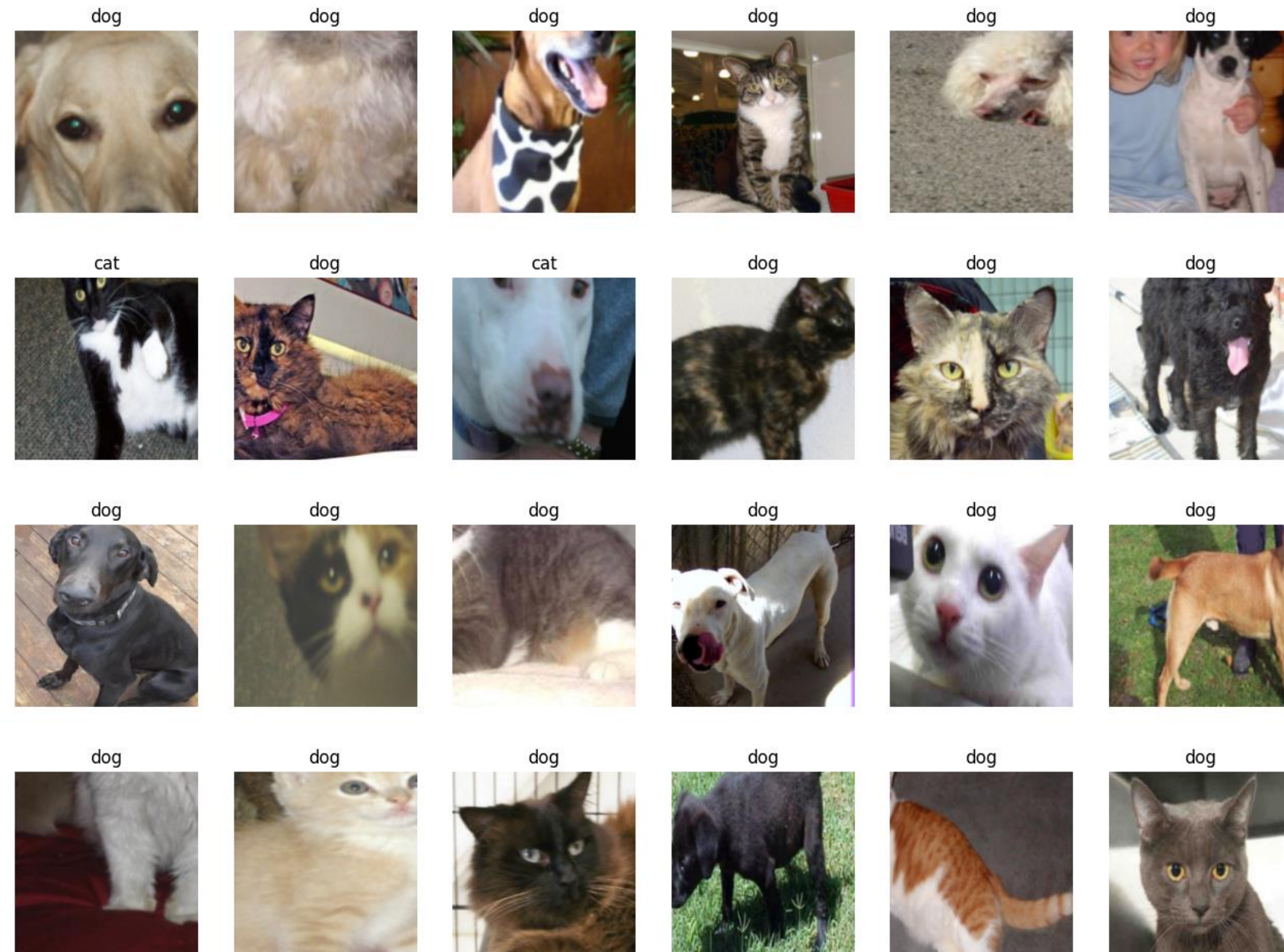
```
classes = {0: 'cat', 1: 'dog'}
fig = plt.figure(figsize=(16,24))
for i in range(24):
    a = fig.add_subplot(4,6,i+1)
    a.set_title(classes[labels[i].item()])
    a.axis('off')
    a.imshow(np.transpose(samples[i].cpu().numpy(), (1,2,0)))
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

오류 예방

- Invalid shape (3,224,224) for image data → **transpose 누락**
- 정규화(Normalize) 추가한 경우엔 **역정규화** 후 표시
- GPU 텐서는 `img.cpu().numpy()`로 변환
- `samples.shape`로 배치 크기·축 확인 (예: [32, 3, 224, 224])

5.3.1 특성 추출 기법

*학습용 배치 24장 미리보기 (PyTorch → Matplotlib)



5.3.1 특성 추출 기법

3. 사전훈련 백본 불러오기

```
import torchvision.models as models  
  
resnet18 = models.resnet18(pretrained=True)
```

- ResNet18을 **사전훈련 가중치**로 불러옴
- 같은 원리로 다른 백본도 교체 가능

5.3.1 특성 추출 기법

4. 백본 동결(Feature Extractor로 사용)

```
def set_parameter_requires_grad(model, feature_extracting=True):  
    if feature_extracting:  
        for param in model.parameters():  
            param.requires_grad = False
```

```
set_parameter_requires_grad(resnet18)
```

- 합성곱/풀링 파트는 동결(requires_grad=False)
- 특성추출기만 재사용 → 속도/일반화 ↑

5. 분류기(FC) 교체

```
resnet18.fc = nn.Linear(512, 2) # FC 교체
```

- in_features 자동 추출 → 클래스 수에 맞춰 FC 교체

5.3.1 특성 추출 기법

*점검: 어떤 파라미터가 학습되나? + 구조 확인

```
for name, param in resnet18.named_parameters():  
    if param.requires_grad:  
        print(name, param.data)
```

```
fc.weight tensor([[ -0.0114, -0.0050, -0.0326, ..., -0.0101,  0.0004, -0.0050],  
                  [ 0.0007, -0.0352, -0.0229, ..., -0.0244,  0.0119,  0.0105]])  
fc.bias tensor([-0.0114,  0.0291])
```

- named_parameters()로 **학습 대상** 확인
- 모델 요약/파라미터 수로 **sanity check**

5.3.1 특성 추출 기법

6. 학습 함수/옵티마 설정/훈련 로그

① train_model() 학습 루프

```
def train_model(model, dataloaders, criterion, optimizer, device, num_epochs=13, is_train=True):
    since = time.time()    # 컴퓨터의 현재 시간을 구하는 함수
    acc_history = []        # 정확도/오차 히스토리
    loss_history = []
    best_acc = 0.0

    for epoch in range(num_epochs): # 에포크(13)만큼 반복
        print('Epoch {}/{}'.format(epoch, num_epochs - 1))
        print('-' * 10)

        running_loss = 0.0
        running_corrects = 0

        for inputs, labels in dataloaders: # 데이터로더에 전달된 데이터만큼 반복
            inputs = inputs.to(device)
            labels = labels.to(device)

            model.to(device)
            optimizer.zero_grad()        # 기울기를 0으로 설정
            outputs = model(inputs)        # 순전파 학습
            loss = criterion(outputs, labels) # 손실값 계산
            _, preds = torch.max(outputs, 1) # 예측 클래스 산출

            loss.backward()                # 역전파 학습
            optimizer.step()

            # 출력 결과와 레이블의 오차를 계산한 값을 누적하여 저장
            running_loss += loss.item() * inputs.size(0)
            # 출력 결과와 레이블이 동일한지 확인한 결과를 누적하여 저장
            running_corrects += torch.sum(preds == labels.data)

        # 평균 오차 계산
        epoch_loss = running_loss / len(dataloaders.dataset)
        # 평균 정확도 계산
        epoch_acc = running_corrects.double() / len(dataloaders.dataset)

        print('Loss: {:.4f} Acc: {:.4f}'.format(epoch_loss, epoch_acc))

        if epoch_acc > best_acc:
            best_acc = epoch_acc

    acc_history.append(epoch_acc.item())
    loss_history.append(epoch_loss)
```

5.3.1 특성 추출 기법

②체크포인트 저장 & 옵티마/손실/디바이스 설정

(슬라이드 1의 반복문 끝에 이어짐)

```
# 에포크 손실 기록
loss_history.append(epoch_loss)

# 모델 지식을 위해 저장해 둡니다(에포크별 체크포인트)
torch.save(model.state_dict(), os.path.join('/content/drive/MyDrive/catanddog', '{0:0=2d}.pth'.format(epoch)))

# 학습 시간(분:초) 계산/출력
time_elapsed = time.time() - since
print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))
print('Best Acc: {:.4f}'.format(best_acc))
return acc_history, loss_history
```

파라미터 학습 결과를 옵티마이저에 전달

```
params_to_update = []
for name,param in resnet18.named_parameters():
    if param.requires_grad == True:
        params_to_update.append(param) # 파라미터 학습 결과를 저장
        print("###",name)

optimizer = optim.Adam(params_to_update) # 학습 결과를 옵티마이저에 전달
```

5.3.1 특성 추출 기법

모델 학습 (장치/손실/호출)

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss() # 손실 함수 지정
train_acc_hist, train_loss_hist = train_model(
    resnet18,          # 모델
    train_loader,      # 학습 데이터
    criterion,         # 손실 함수
    optimizer,         # 옵티마이저
    device             # 장치 (CPU 혹은 GPU)
)
```


5.3.1 특성 추출 기법

훈련 로그 해석

해석

- 손실 ↓ / 정확도 ↑ : 전이학습 + 분류기 학습이 안정적으로 수렴
- **Best Acc ≈ 0.937**: 에포크 중 최고 성능 스냅샷 존재
- **주의**: 훈련 데이터 기준 지표이므로, 테스트/검증 데이터로 일반화 성능을 별도 확인해야 함

Epoch 0/12 ----- Loss: 0.5614 Acc: 0.7221	Epoch 7/12 ----- Loss: 0.2067 Acc: 0.9143
Epoch 1/12 ----- Loss: 0.4387 Acc: 0.7948	Epoch 8/12 ----- Loss: 0.1965 Acc: 0.9221
Epoch 2/12 ----- Loss: 0.3055 Acc: 0.8961	Epoch 9/12 ----- Loss: 0.2309 Acc: 0.9091
Epoch 3/12 ----- Loss: 0.3092 Acc: 0.8805	Epoch 10/12 ----- Loss: 0.3047 Acc: 0.8494
Epoch 4/12 ----- Loss: 0.4092 Acc: 0.8000	Epoch 11/12 ----- Loss: 0.1858 Acc: 0.9377
Epoch 5/12 ----- Loss: 0.3564 Acc: 0.8312	Epoch 12/12 ----- Loss: 0.1985 Acc: 0.9169
Epoch 6/12 ----- Loss: 0.2051 Acc: 0.9377	Training complete in 6m 27s Best Acc: 0.937662

5.3.1 특성 추출 기법

7. 테스트 평가 함수 생성 eval_model()

```
def eval_model(model, dataloaders, device):
    since = time.time()
    acc_history = []
    best_acc = 0.0

    saved_models = glob.glob('../chap05/data/catanddog/' + '*.pth') # ①
    saved_models.sort() # 확장자 .pth 파일들을 정렬
    print('saved_model', saved_models)

    for model_path in saved_models:
        print('Loading model', model_path)
        model.load_state_dict(torch.load(model_path)) # 체크포인트 로드
        model.eval() # 평가 모드
        model.to(device)
        running_corrects = 0

        for inputs, labels in dataloaders: # 테스트 반복
            inputs = inputs.to(device)
            labels = labels.to(device)

            with torch.no_grad(): # autograd를 사용하지 않는다는 의미
                outputs = model(inputs) # 데이터를 모델에 적용한 결과 outputs에 저장

            _, preds = torch.max(outputs.data, 1) # ②
            preds[preds >= 0.5] = 1 # ③ torch.max로 출력된 값이 0.5보다 크면 양클래스 예측
            preds[preds < 0.5] = 0 # ③ torch.max로 출력된 값이 0.5보다 작으면 음클래스 예측

            # 모델의 예측 결과가 정답(레이블)과 일치하는 것들의 개수를 누적
            running_corrects += preds.eq(labels).sum()

        # 테스트 데이터의 정확도 계산
        epoch_acc = running_corrects.double() / len(dataloaders.dataset)
        print('Acc: {:.4f}'.format(epoch_acc))

        if epoch_acc > best_acc:
            best_acc = epoch_acc

        acc_history.append(epoch_acc.item())

    time_elapsed = time.time() - since
    print('Validation complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))
    print('Best Acc: {:.6f}'.format(best_acc))
    return acc_history # 계산된 정확도 반환
```

5.3.1 특성 추출 기법

8. 테스트 데이터 평가 실행 & 로그 해석

```
val_acc_hist = eval_model(resnet18, test_loader, device)
```

해석

- 체크포인트별 정확도를 비교해 최고 스냅샷(예: ~0.949) 선택
- 훈련셋 최고치(~0.938) 대비 테스트셋도 유사하거나 ↑ → 일반화 양호
- 전이학습(특징 고정 + FC 학습)만으로 짧은 시간에 94~95% 달성
- 다음 단계: 미세조정(fine-tuning), 증강 강화, 스케줄러로 추가 개선

```
Loading model e:/torch/chap05/data/catanddog#00.pth  
Acc: 0.7347
```

```
Loading model e:/torch/chap05/data/catanddog#01.pth  
Acc: 0.8878
```

```
Loading model e:/torch/chap05/data/catanddog#02.pth  
Acc: 0.9184
```

```
Loading model e:/torch/chap05/data/catanddog#03.pth  
Acc: 0.8878
```

```
Loading model e:/torch/chap05/data/catanddog#04.pth  
Acc: 0.9388
```

```
Loading model e:/torch/chap05/data/catanddog#05.pth  
Acc: 0.9286
```

```
Loading model e:/torch/chap05/data/catanddog#06.pth  
Acc: 0.9286
```

```
Loading model e:/torch/chap05/data/catanddog#07.pth  
Acc: 0.9082
```

```
Loading model e:/torch/chap05/data/catanddog#08.pth  
Acc: 0.9388
```

```
Loading model e:/torch/chap05/data/catanddog#09.pth  
Acc: 0.9490
```

```
Loading model e:/torch/chap05/data/catanddog#10.pth  
Acc: 0.9286
```

```
Loading model e:/torch/chap05/data/catanddog#11.pth  
Acc: 0.9286
```

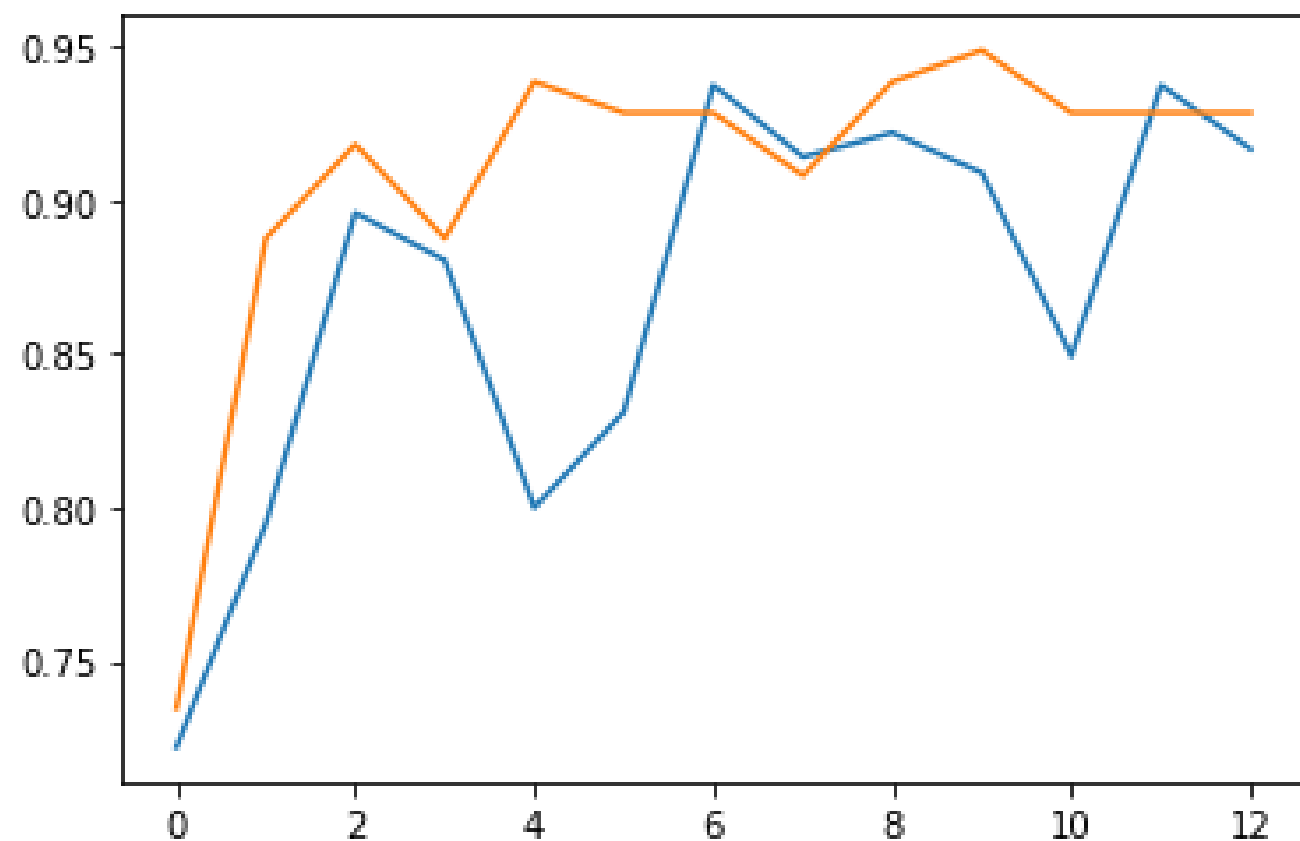
```
Loading model e:/torch/chap05/data/catanddog#12.pth  
Acc: 0.9286
```

```
Validation complete in 1m 52s  
Best Acc: 0.948980
```

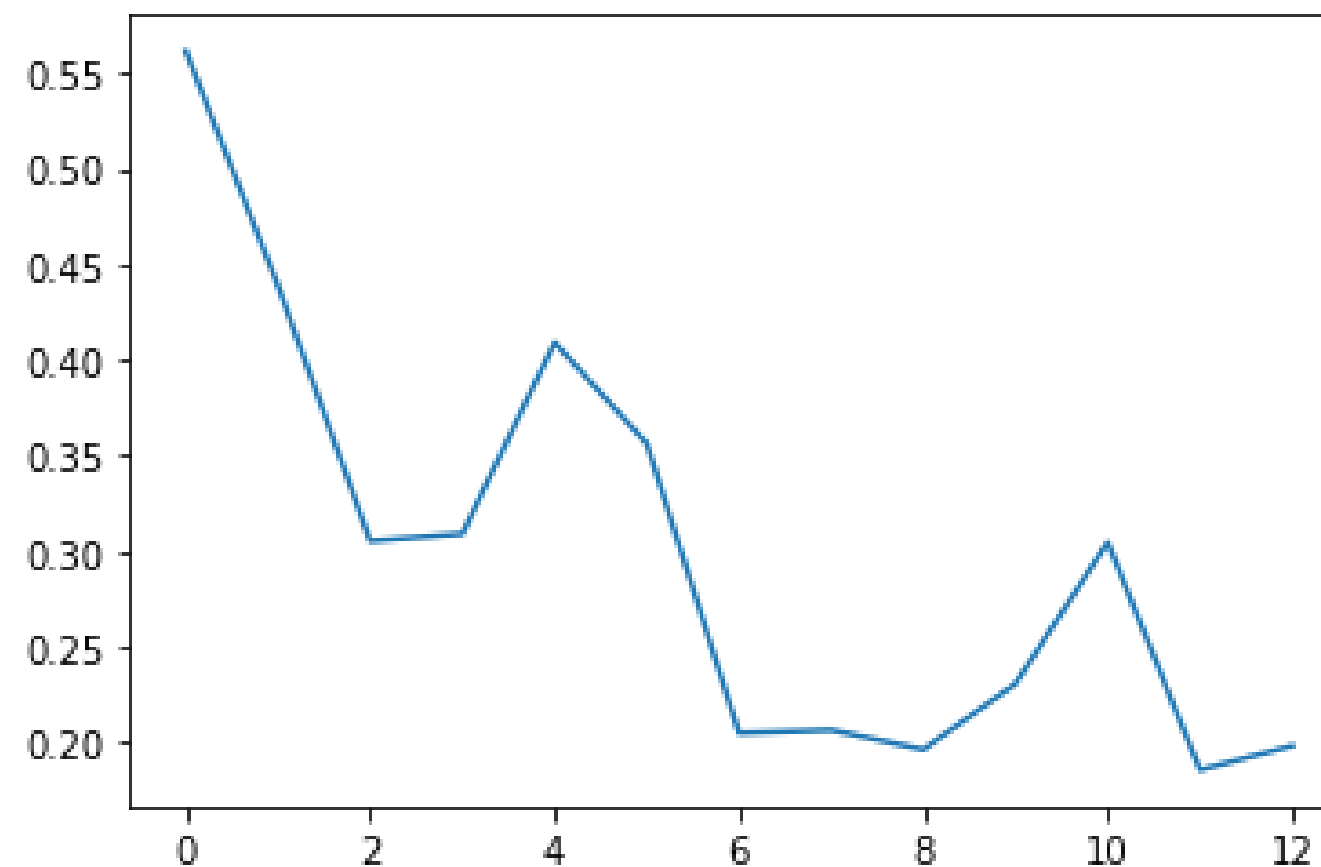
5.3.1 특성 추출 기법

9. 학습 결과 시각화(정확도 & 오차)

```
plt.plot(train_acc_hist) # 훈련 정확도  
plt.plot(val_acc_hist)  # 테스트(검증) 정확도  
plt.show()
```



```
plt.plot(train_loss_hist) # 에포크가 진행될수록 오차가 낮아지는지 확인  
plt.show()
```



5.3.1 특성 추출 기법

10. 예측 이미지 출력 전처리 im_convert()

```
def im_convert(tensor):  
    image=tensor.clone().detach().numpy()      # ① 계산 그래프 분리 + 새 메모리 사본  
    image=image.transpose(1,2,0)              # ② (C,H,W) → (H,W,C)로 축 변환  
    image=image*(np.array((0.5,0.5,0.5))+np.array((0.5,0.5,0.5))) # ③  
    image=image.clip(0,1)                     # ④ 0~1 범위로 값 제한  
    return image
```

① clone().detach()

- clone() → 새 메모리에 복사(계산 그래프는 연결 유지)
- detach() → 계산 그래프에서 분리, 메모리 공유
- clone().detach() → 새 메모리 + 그래프 분리 → numpy() 안전 변환
- (GPU 사용 시) tensor.cpu().clone().detach().numpy() 권장

② transpose(1,2,0) : PyTorch(CHW) → matplotlib(HWC)

③ 역정규화 주의

- 일반적인 역정규화: $image = image * std + mean$
- 교재 예시의 $*(0.5+0.5)$ 는 오타 가능 → 정석은 $image = image * 0.5 + 0.5$

④ clip(0,1) : 표시용으로 0~1 범위에 자르기 (값 튀는 현상 방지)

5.3.1 특성 추출 기법

10. 예측 이미지 출력 전처리 im_convert()

clone / detach 차이

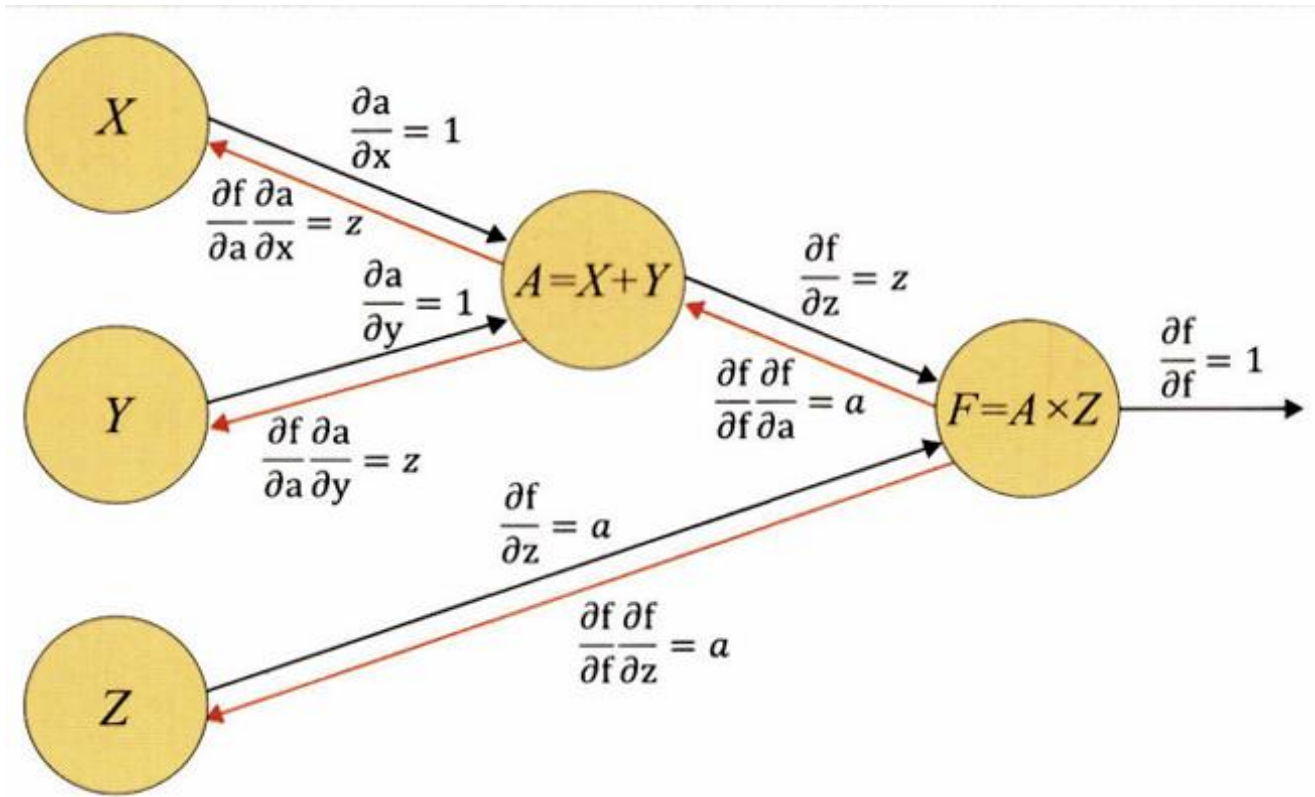
구분	메모리	계산 그래프
tensor.clone()	새로 할당	연결 유지
tensor.detach()	공유	연결 없음
tensor.clone().detach()	새로 할당	연결 없음

5.3.1 특성 추출 기법

*왜 detach가 필요한가?

계산 그래프

- 연산은 노드와 엣지로 이어진 **computational graph**로 기록 → 역전파 시 연쇄 법칙(chain rule)으로 기울기 전파
- 시각화용 텐서는 학습과 무관 → 그래프에서 분리(detach) 해야 불필요한 추적.메모리 사용을 막음

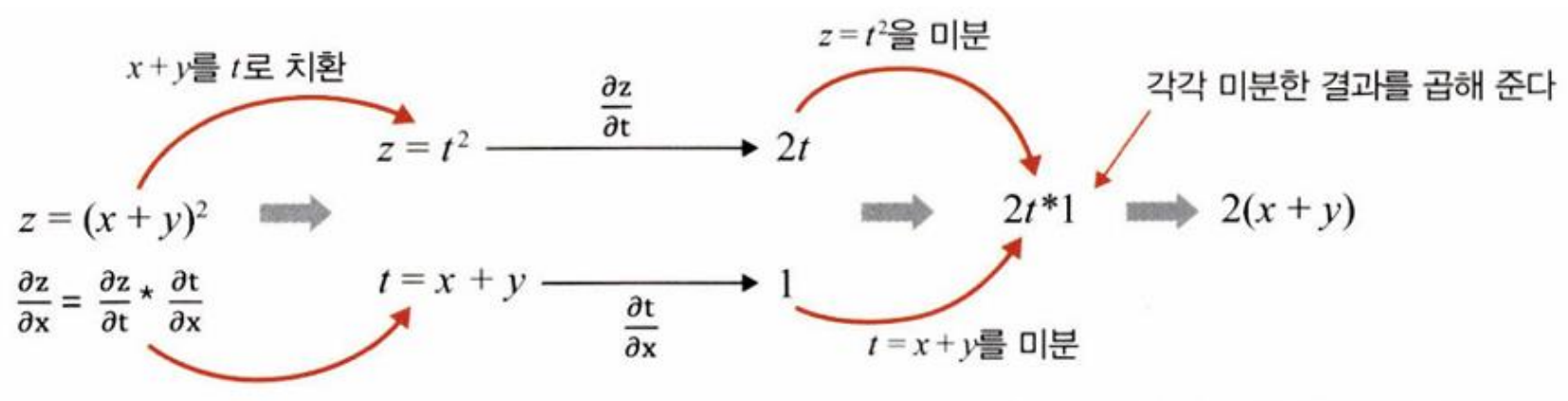


5.3.1 특성 추출 기법

*왜 detach가 필요한가?

연쇄 법칙

• $z = f(x, y) = A(x, y) \times Z$ 처럼 합성함수이면 $\frac{\partial z}{\partial x} = \frac{\partial z}{\partial A} \frac{\partial A}{\partial x}$ 식으로 곱의 법칙으로 전파



- 의미: 주어진 범위를 벗어나는 값을 하한/상한으로 자름
- im_convert()의 image.clip(0,1)은 0~1 사이로 제한하여 화면 표시용으로 안전하게 만듦

5.3.1 특성 추출 기법

11. 테스트 배치에서 20장 예측 시각화 (정답=초록, 오답=빨강)

```
classes = {0:'cat', 1:'dog'} # 개와 고양이 두 개의 라벨

dataiter=iter(test_loader) # ——— 테스트 데이터셋을 가져옵니다.
images, labels = next(dataiter) # ——— 테스트 데이터셋에서 이미지와 레이블을 분리하여 가져옵니다.

output=model(images) # 모델 추론
_,preds=torch.max(output,1) # ——— 최고 확률의 클래스 인덱스

fig=plt.figure(figsize=(25,4))

for idx in np.arange(20):
    ax=fig.add_subplot(2,10,idx+1,xticks=[],yticks=[]) # ①
    plt.imshow(im_convert(images[idx])) # ——— im_convert로 시각화 전처리
    a.set_title(classes[labels[i].item()]) # 정답 라벨(아래줄)

    ax.set_title("{}({})".format( # ② 제목: 예측(정답)
        str(classes[preds[idx].item()]),
        str(classes[labels[idx].item()])))
    color=("green" if preds[idx]==labels[idx] else "red")) # ——— 맞으면 초록, 틀리면 빨강
```

매개변수 설명

① add_subplot(2, 10, idx+1, xticks=[], yticks=[])

a) **행=2**, b) **열=10** → 2×10 그리드

c) **idx+1** → 좌→우, 위→아래 순으로 채움

d) **xticks/yticks=[]** → 눈금 삭제(이미지 깔끔)

② classes[preds[idx].item()]는 **예측 클래스 이름**,
classes[labels[idx].item()]는 **정답 클래스 이름**

③ subplots_adjust(bottom, top, hspace)로 **여백/간격** 미세 조정

5.3.1 특성 추출 기법

11. 테스트 배치에서 20장 예측 시각화 (정답=초록, 오답=빨강)



해석

- 초록 제목: **예측=정답** (예: dog(dog))
- 빨간 제목: **오분류** (예: dog(cat))
- 오분류 패턴을 관찰:
 - 배경/조명/블러/부분가림에 약함
 - 강아지·고양이의 자세/크롭에 따라 경계가 애매한 경우 존재
- 정량 요약: 테스트 정확도 ~ 94-95%(앞 슬라이드)와 일관

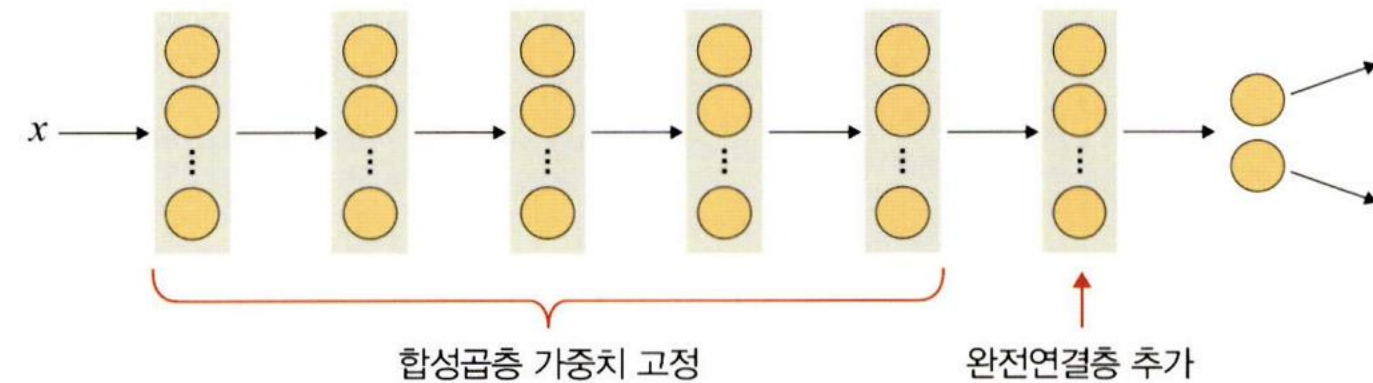
5.3.1 특성 추출 기법

1. ResNet18 다운로드

```
resnet18 = models.resnet18(pretrained=True)
```

```
def set_parameter_requires_grad(model, feature_extracting=True):  
    if feature_extracting:  
        for param in model.parameters():  
            param.requires_grad = False # 모델의 일부를 고정하고 나머지 학습  
  
set_parameter_requires_grad(resnet18)
```

```
resnet18.fc = nn.Linear(512, 2) # ResNet18의 마지막 부분에 완전연결층 추가
```



5.3.1 특성 추출 기법

2. 모델 학습 준비_모델의 객체 생성 및 손실 함수 정의

```
model = models.resnet18(pretrained = True) # 모델 객체 생성

for param in model.parameters():
    param.requires_grad = False # 모델의 합성곱층 가중치 고정

model.fc = torch.nn.Linear(512, 2)
for param in model.fc.parameters():
    param.requires_grad = True # 모델의 완전곱층은 학습

optimizer = torch.optim.Adam(model.fc.parameters())
cost = torch.nn.CrossEntropyLoss() # 손실함수 정의
print(model)
```

5.3.1 특성 추출 기법

3. 모델 학습

```
def train_model(model, dataloaders, criterion, optimizer, device, num_epochs=13, is_train=True):
    since = time.time() # 컴퓨터의 현재 시각을 구하는 함수
    acc_history = []
    loss_history = []
    best_acc = 0.0

    for epoch in range(num_epochs): # 에포크만큼(13번) 반복
        print('Epoch {}/{}'.format(epoch, num_epochs - 1))
        print('-' * 10)

        running_loss = 0.0
        running_corrects = 0

        for inputs, labels in dataloaders: # 데이터로드에 전달된 데이터만큼 반복
            inputs = inputs.to(device)
            labels = labels.to(device)

            model.to(device)
            optimizer.zero_grad() # 기울기 0 설정
            outputs = model(inputs) # 순전파 학습
            loss = criterion(outputs, labels)
            _, preds = torch.max(outputs, 1)
            loss.backward() # 역전파 학습
            optimizer.step()

            running_loss += loss.item() * inputs.size(0) # 출력 결과/레이블의 오차 계산 결과 누적해 저장
            running_corrects += torch.sum(preds == labels.data) # 출력 결과/레이블의 동일한지 확인한 결과 누적해 저장

        #평균 오차,
        #정확도 계산
        epoch_loss = running_loss / len(dataloaders.dataset)
        epoch_acc = running_corrects.double() / len(dataloaders.dataset)

        print('Loss: {:.4f} Acc: {:.4f}'.format(epoch_loss, epoch_acc))

        if epoch_acc > best_acc:
            best_acc = epoch_acc

        acc_history.append(epoch_acc.item())
        loss_history.append(epoch_loss)
        torch.save(model.state_dict(), os.path.join('../chap05/data/catanddog/', '{0:0=2d}.pth'.format(epoch))) # 모델 재사용을 위한 저장
        print()

    time_elapsed = time.time() - since # 실행(학습) 시간 계산
    print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))
    print('Best Acc: {:.4f}'.format(best_acc))
    return acc_history, loss_history # 정확도/오차 반환
```


5.3.1 특성 추출 기법

3. 모델 학습

파라미터의 학습 결과를 옵티마이저에 전달

```
params_to_update = []
for name,param in resnet18.named_parameters():
    if param.requires_grad == True:
        params_to_update.append(param) # 파라미터 학습 결과 저장
        print("\t",name)

optimizer = optim.Adam(params_to_update) # 학습 결과 옵티마이저에 전달
```

모델 학습

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss() # 손실함수 지정
train_acc_hist, train_loss_hist = train_model(resnet18, train_loader, criterion, optimizer, device)
```

(중략)

Epoch 10/12

Loss: 0.3047 Acc: 0.8494

Epoch 11/12

Loss: 0.1858 Acc: 0.9377

Epoch 12/12

Loss: 0.1985 Acc: 0.9169

Training complete in 6m 27s

Best Acc: 0.937662

93%로 꽤 높은 정확도 -> 훈련 데이터로 학습이 잘 되었음

5.3.1 특성 추출 기법

4. 테스트 데이터를 활용한 모델 정확도 측정

테스트 데이터 호출 및 전처리

```
test_path = '../chap05/data/catanddog/test'

transform = transforms.Compose(
    [
        transforms.Resize(224),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
    ])
test_dataset = torchvision.datasets.ImageFolder(
    root=test_path,
    transform=transform
)
test_loader = torch.utils.data.DataLoader(
    test_dataset,
    batch_size=32,
    num_workers=1,
    shuffle=True
)

print(len(test_dataset))
```

5.3.1 특성 추출 기법

4. 테스트 데이터를 활용한 모델 정확도 측정

테스트 데이터 평가 함수 생성

```
def eval_model(model, dataloaders, device):
    since = time.time()
    acc_history = []
    best_acc = 0.0

    saved_models = glob.glob('../chap05/data/catanddog/' + '*.pth')
    saved_models.sort()
    print('saved_model', saved_models)

    for model_path in saved_models:
        print('Loading model', model_path)

        model.load_state_dict(torch.load(model_path))
        model.eval()
        model.to(device)
        running_corrects = 0

        for inputs, labels in dataloaders:
            inputs = inputs.to(device)
            labels = labels.to(device)

            with torch.no_grad():
                outputs = model(inputs)

            _, preds = torch.max(outputs.data, 1)
            preds[preds >= 0.5] = 1
            preds[preds < 0.5] = 0
            running_corrects += preds.eq(labels.cpu()).int().sum()

        epoch_acc = running_corrects.double() / len(dataloaders.dataset)
        print('Acc: {:.4f}'.format(epoch_acc))

        if epoch_acc > best_acc:
            best_acc = epoch_acc

        acc_history.append(epoch_acc.item())
        print()

    time_elapsed = time.time() - since
    print('Validation complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))
    print('Best Acc: {:.4f}'.format(best_acc))

    return acc_history
```

glob: 현재 디렉터리에서 원하는 파일들만 추출해서 가져올 때 사용

torch.max: 주어진 텐서 배열의 최댓값이 들어 있는 index 반환

preds.eq(labels): preds 배열과 labels가 일치하는지 검사

5.3.1 특성 추출 기법

4. 테스트 데이터를 활용한 모델 정확도 측정

성능(정확도) 측정

```
val_acc_hist = eval_model(resnet18, test_loader, device)
```

(중략)

Loading model e:/torch/chap05/data/catanddog\10.pth
Acc: 0.9286

Loading model e:/torch/chap05/data/catanddog\11.pth
Acc: 0.9286

Loading model e:/torch/chap05/data/catanddog\12.pth
Acc: 0.9286

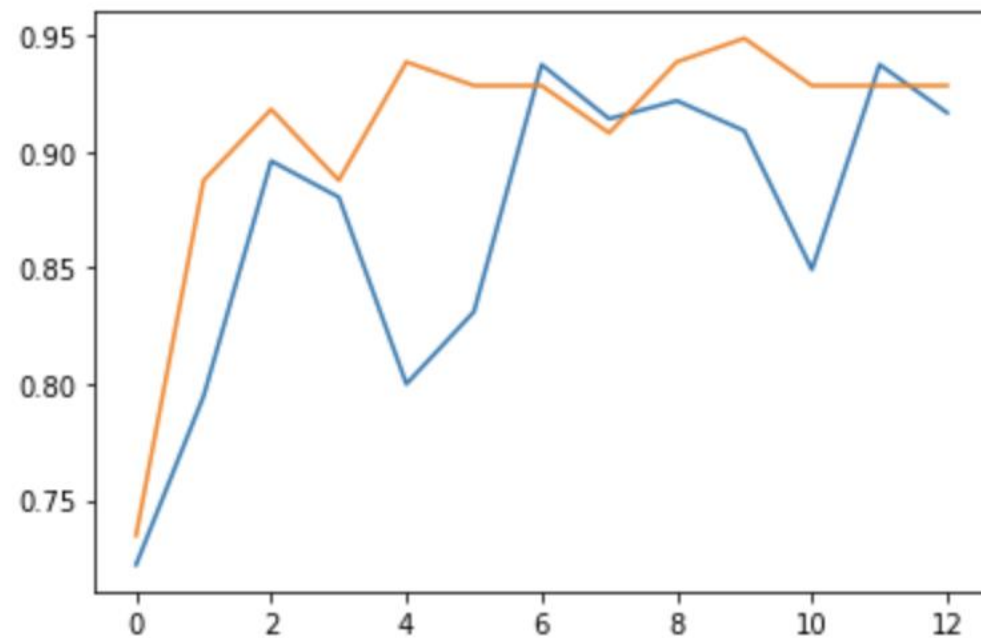
Validation complete in 1m 52s
Best Acc: 0.948980 # 94%로 높은 정확도!

5.3.1 특성 추출 기법

5. 학습 결과 시각화

정확도

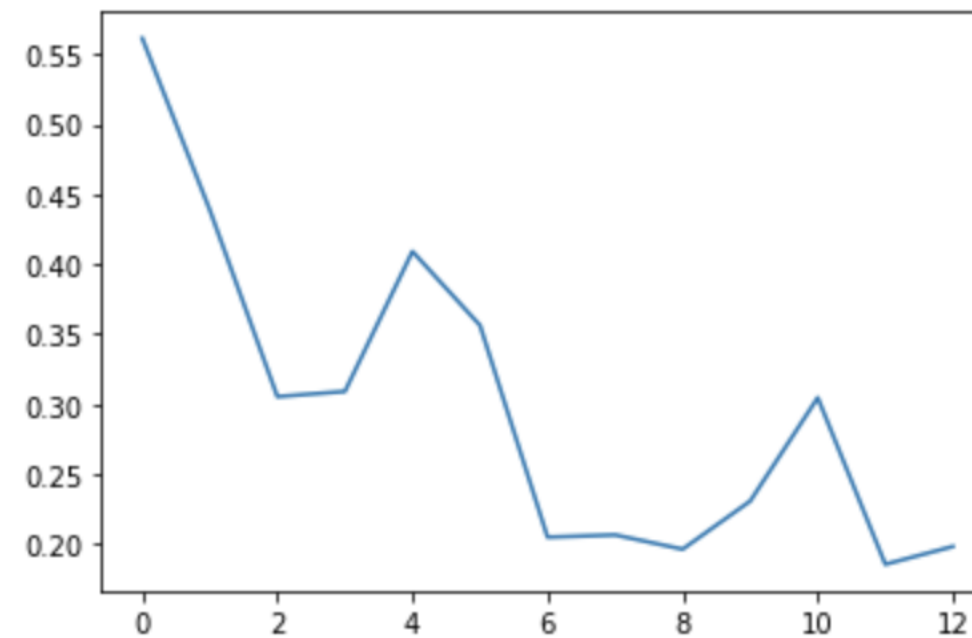
```
plt.plot(train_acc_hist)
plt.plot(val_acc_hist)
plt.show()
```



=> 에포크가 진행될 수록 정확도 높아짐

훈련 데이터에 대한 오차

```
plt.plot(train_loss_hist)
plt.show()
```



=> 에포크가 진행될 수록 오차 낮아짐

5.3.1 특성 추출 기법

6. 실제 데이터로 성능 확인

예측 이미지 출력을 위한 전처리 함수

```
def im_convert(tensor):  
    image=tensor.clone().detach().numpy()    # tensor.clone() : 기존 텐서의 내용을 복사한 텐서를 생성  
    image=image.transpose(1,2,0)             # detach() : 기존 텐서에서 기울기가 전파되지 않는 텐서  
    image=image*(np.array((0.5,0.5,0.5))+np.array((0.5,0.5,0.5)))  
    image=image.clip(0,1) # clip() : 입력 값을 특정 범위로 제한시킬 때 사용  
    return image                               (여기에서는 image 데이터를 0과 1 사이의 값으로 제한하기 위해 사용)
```

=> 기울기에 영향을 주지 않는 복사된 텐서 생성!

5.3.1 특성 추출 기법

6. 실제 데이터로 성능 확인

예측 결과 출력

```
classes = {0:'cat', 1:'dog'} # 개, 고양이 2개의 레이블
```

```
dataiter=iter(test_loader) # 테스트 데이터셋 로드
```

```
images,labels=dataiter.next() # 테스트 데이터셋에서 이미지/레이블 분리
```

```
output=model(images)
```

```
_,preds=torch.max(output,1)
```

```
fig=plt.figure(figsize=(25,4))
```

```
for idx in np.arange(20):
```

```
    ax=fig.add_subplot(2,10,idx+1,xticks=[],yticks=[])
```

```
    plt.imshow(im_convert(images[idx])) # 앞서 정의한 im_convert 함수 이용해 이미지 출력
```

```
    a.set_title(classes[labels[i].item()])
```

```
    ax.set_title("{}({})".format(str(classes[preds[idx].item()]),str(classes[labels[idx].item()])),
```

```
        color=("green" if preds[idx]==labels[idx] else "red"))
```

```
plt.show()
```

```
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

행 열 인덱스 틱 삭제
ax=fig.add_subplot(2,10,idx+1,xticks=[],yticks=[])



초록색: 정확한 예측 / 빨간색: 잘못된 예측
=> 예측이 정확하지 않음
=> 훈련데이터와 에포크 수를 늘려야 함

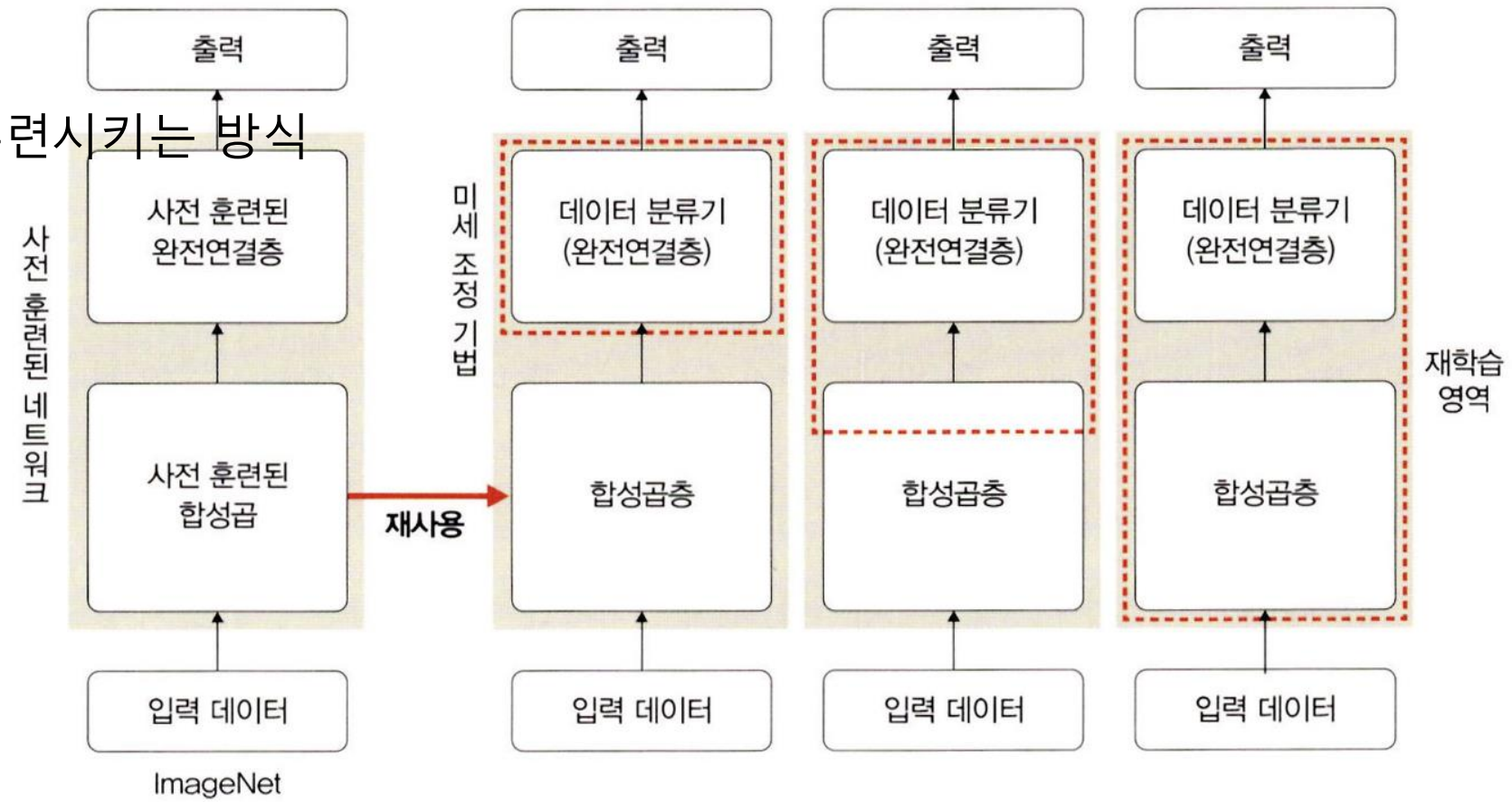
5.3.2 미세 조정 기법

정리

- 사전 훈련된 모델/ 합성곱층/데이터 분류기의 가중치를 업데이트해 훈련시키는 방식
-> 사전 학습된 모델을 목적에 맞게 재학습시키거나 학습된 가중치의 일부를 재학습시키는 것!
- 파라미터에 큰 변화를 주게 되면 과적합 문제가 발생할 수 있음
-> 정교하고 미세한 파라미터 업데이트 필요

미세 조정 기법 전략

- 데이터셋 크기 / 사전 훈련된 모델과의 유사도에 따른 학습 전략



데이터셋 크기 \ 유사도	유사도	
	↑	↓
↑	합성곱층 뒷부분(완전연결층과 가까운 부분)과 데이터 분류기 학습	모델 전체 재학습
↓	데이터 분류기만 학습	합성곱층의 일부분과 데이터 분류기 재학습

5.4 설명 가능한 CNN



5.4 설명 가능한 CNN

▼ 그림 5-45 CNN의 블랙박스



- 딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술
- CNN은 내부 동작 설명이 어려움 -> 결과 신뢰성 확보가 어려움
- 시각화 방법: 필터 시각화, 특성 맵 시각화

5.4.1 특성 맵 시각화

실습 준비

라이브러리 설치

```
> pip install pillow
```

라이브러리 호출

코드 5-30 필요한 라이브러리 호출

```
import matplotlib.pyplot as plt
from PIL import Image
import cv2
import torch
import torch.nn.functional as F
import torch.nn as nn
from torchvision.transforms import ToTensor
import torchvision

import torchvision.transforms as transforms
import torchvision.models as models

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```


5.4.1 특성 맵 시각화

모델 생성

- 합성곱층 13개
- 완전연결층 2개
- 활성화 함수 ReLU 사용

코드 5-31 설명 가능한 네트워크 생성

```
class XAI(torch.nn.Module):
    def __init__(self, num_classes=2):
        super(XAI, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True), ----- inplace=True는 기존의 데이터를 연산의
            nn.Dropout(0.3),                  결맞음으로 대체하는 것을 의미
            nn.Conv2d(64, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(128, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(128, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
```

```
nn.MaxPool2d(kernel_size=2, stride=2),

nn.Conv2d(256, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),
```

```
nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),
```

```
)
self.classifier = nn.Sequential(
    nn.Linear(512, 512, bias=False),
    nn.Dropout(0.5),
    nn.BatchNorm1d(512),
    nn.ReLU(inplace=True),
    nn.Dropout(0.5),
    nn.Linear(512, num_classes)
)
```

```
def forward(self, x):
    x = self.features(x)
    x = x.view(-1, 512)
    x = self.classifier(x)
    return F.log_softmax(x) ----- ①
```

5.4.1 특성 맵 시각화

```
def forward(self, x):  
    x = self.features(x)  
    x = x.view(-1, 512)  
    x = self.classifier(x)  
    return F.log_softmax(x) ----- ①
```

F.log_softmax(x)

- 소프트맥스 결과에 log 값을 취한 연산
- 목적: 신경망 말단 결괏값을 확률 개념으로 해석
- 소프트맥스는 기울기 소멸 문제에 취약

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1} e^{x_j}}$$

$$\begin{aligned}\text{logsoftmax} &= \log\left(\frac{e^{x_i}}{\sum_{j=1} e^{x_j}}\right) \\ &= x_i - \log\left(\sum_{j=1} e^{x_j}\right)\end{aligned}$$

5.4.1 특성 맵 시각화

모델 객체화 및 장치 할당

코드 5-32 모델 객체화

```
model = XAI() ----- model이라는 이름의 객체를 생성
model.to(device) ----- model을 장치(CPU 혹은 GPU)에 할당
model.eval() ----- 테스트 데이터에 대한 모델 평가 용도로 사용
```

- 앞서 생성한 모델을 `model = XAI()`로 객체화
- 장치(CPU or GPU)에 할당
- 모델을 print한 구조 결과 예시 ->

모델 구조 출력

```
XAI(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), bias=False)
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.3, inplace=False)
    (4): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU(inplace=True)
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): ReLU(inplace=True)
    (11): Dropout(p=0.4, inplace=False)
    (12): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (13): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (14): ReLU(inplace=True)
```


5.4.1 특성 맵 시각화

특성 맵 확인 함수 정의

코드 5-33 특성 맵을 확인하기 위한 클래스 정의

```
class LayerActivations:
    features = []
    def __init__(self, model, layer_num):
        self.hook = model[layer_num].register_forward_hook(self.hook_fn) ----- ①

    def hook_fn(self, module, input, output):
        self.features = output.detach().numpy()

    def remove(self): ----- hook 삭제
        self.hook.remove()
```

- 목표: 특성 합성곱층의 입력/출력을 확보하여 특성 맵 값 확인
- 배경: 합성곱층은 이미지와 필터 연산 결과 -> 특성 맵
- 예시 대상: (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), bias=False)
- Register_forward_hook
- 파이토치 혹은 기능: 매 계층마다 print 없이 활성화 및 기울기 값 확인 가능
- 목적: 순전파 중 각 모듈의 입력/출력 가져옴

5.4.1 특성 맵 시각화

중간 변수 기울기 훅 예시

```
import torch
x = torch.Tensor([0,1,2,3]).requires_grad_()
y = torch.Tensor([4,5,6,7]).requires_grad_()
w = torch.Tensor([1,2,3,4]).requires_grad_()
z = x + y;
o = w.matmul(z)
o.backward()
print(x.grad, y.grad, z.grad, w.grad, o.grad)
```

```
tensor([2., 3., 4., 5.]) tensor([2., 3., 4., 5.]) None tensor([ 4.,  6.,  8., 10.]) None
```

- o, x 같은 중간 변수는 기본적으로 기울기 미저장
- Hook으로 중간 결괏값들을 확인할 수 있음

5.4.1 특성 맵 시각화

특성 맵에서 사용할 이미지 호출

코드 5-34 이미지 호출

```
img = cv2.imread("../chap05/data/cat.jpg")
plt.imshow(img)
img = cv2.resize(img, (100,100), interpolation=cv2.INTER_LINEAR) ----- ①
img = ToTensor()(img).unsqueeze(0) ----- ②
print(img.shape)
```

cv2.resize(img, (100,100), interpolation=cv2.INTER_LINEAR)

① ② ③

① 첫 번째 파라미터: 변경할 이미지 파일

② 두 번째 파라미터: 변경될 이미지 크기를 (너비, 높이)로 지정

③ interpolation: 보간법

보간법: 크기 변경 시 존재하지 않는 픽셀에
근사 함수 $f(x, y)$ 적용
-> 새 픽셀 값 할당
or 압축 시 새 값 할당

5.4.1 특성 맵 시각화

텐서 변환 및 차원 확장

```
import torch

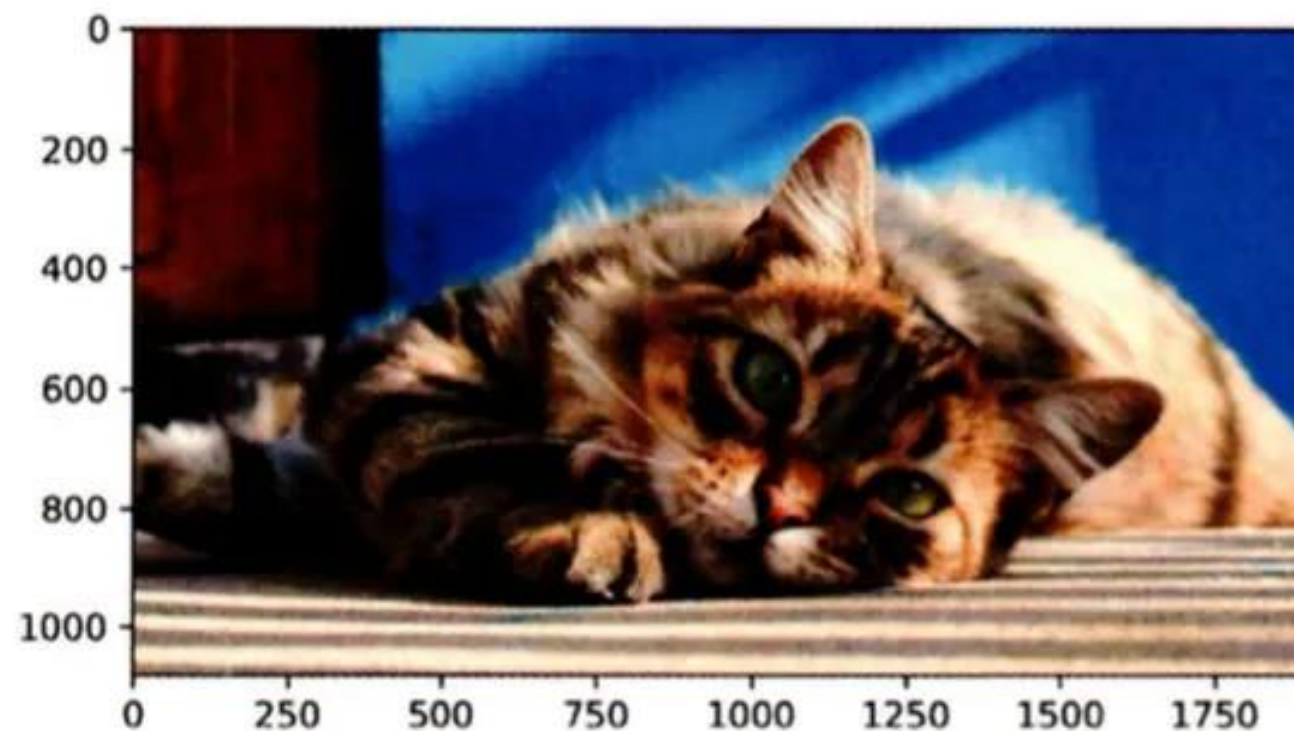
x1 = torch.rand(3, 10, 64)

x2 = x1.unsqueeze(dim=0) ----- [3, 10, 64] -> [1, 3, 10, 64]
print(x2.shape)
print('-----')
x3 = x1.unsqueeze(dim=1) ----- [3, 10, 64] -> [3, 1, 10, 64]
print(x3.shape)
```

```
torch.Size([1, 3, 10, 64])
```

```
torch.Size([3, 1, 10, 64])
```

▼ 그림 5-46 예제에서 사용할 이미지



5.4.1 특성 맵 시각화

첫 합성곱층 특성 맵 확인

코드 5-35 (0): Conv2d 특성 맵 확인

```
result = LayerActivations(model.features, 0) ----- 0번째 Conv2d 특성 맵 확인
```

```
model(img)
activations = result.features
```

```
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()
```

▼ 그림 5-47 첫 번째 계층에서 특성 맵



대상: 코드 5-32의 (0) Conv2d(3, 64, k=3, s=1, bias=False)
- 입력층과 가까운 계층은 입력 이미지 형태가 많이 유지

5.4.1 특성 맵 시각화

20번째 계층 특성 맵

코드 5-37 20번째 계층에 대한 특성 맵

```
result = LayerActivations(model.features, 20) ----- 20번째 Conv2d 특성 맵 확인

model(img)
activations = result.features
```

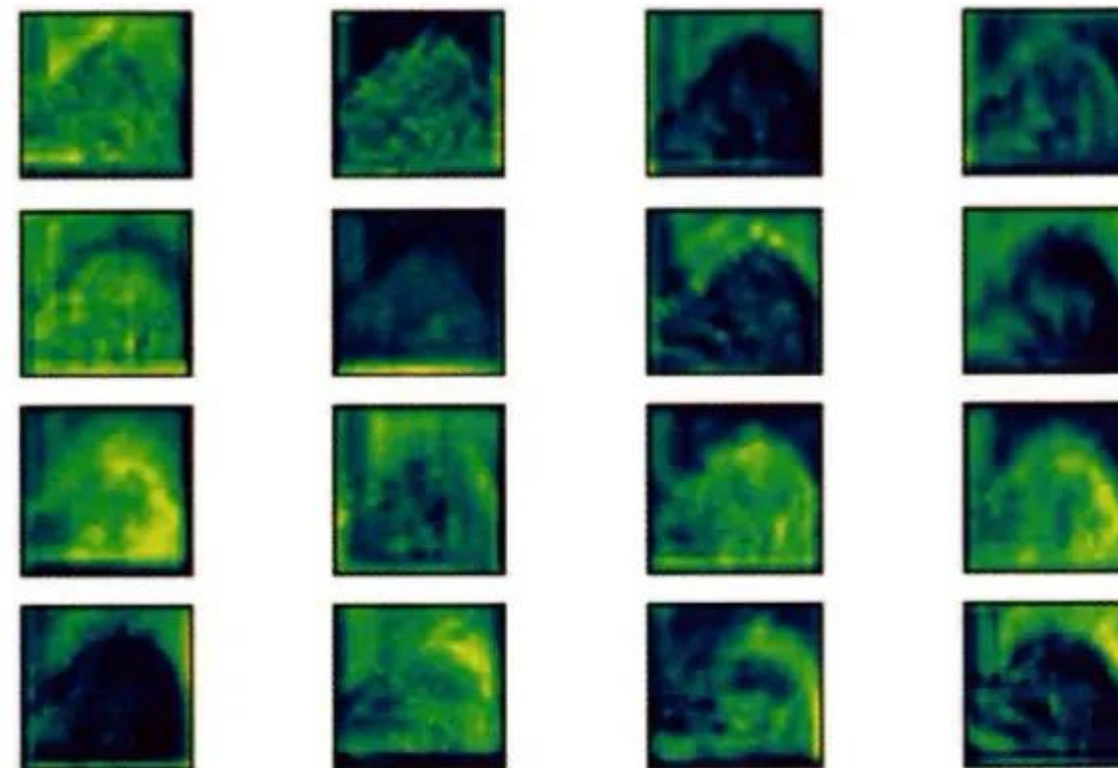
코드 5-38 특성 맵 확인

```
fig, axes = plt.subplots(4, 4)

fig = plt.figure(figsize=(12,8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])

plt.show()
```

▼ 그림 5-48 20번째 계층에서 특성 맵



5.4.1 특성 맵 시각화

40번째 계층 특성 맵

코드 5-39 40번째 계층에 대한 특성 맵

```
result = LayerActivations(model.features, 40) ----- 40번째 Conv2d 특성 맵 확인

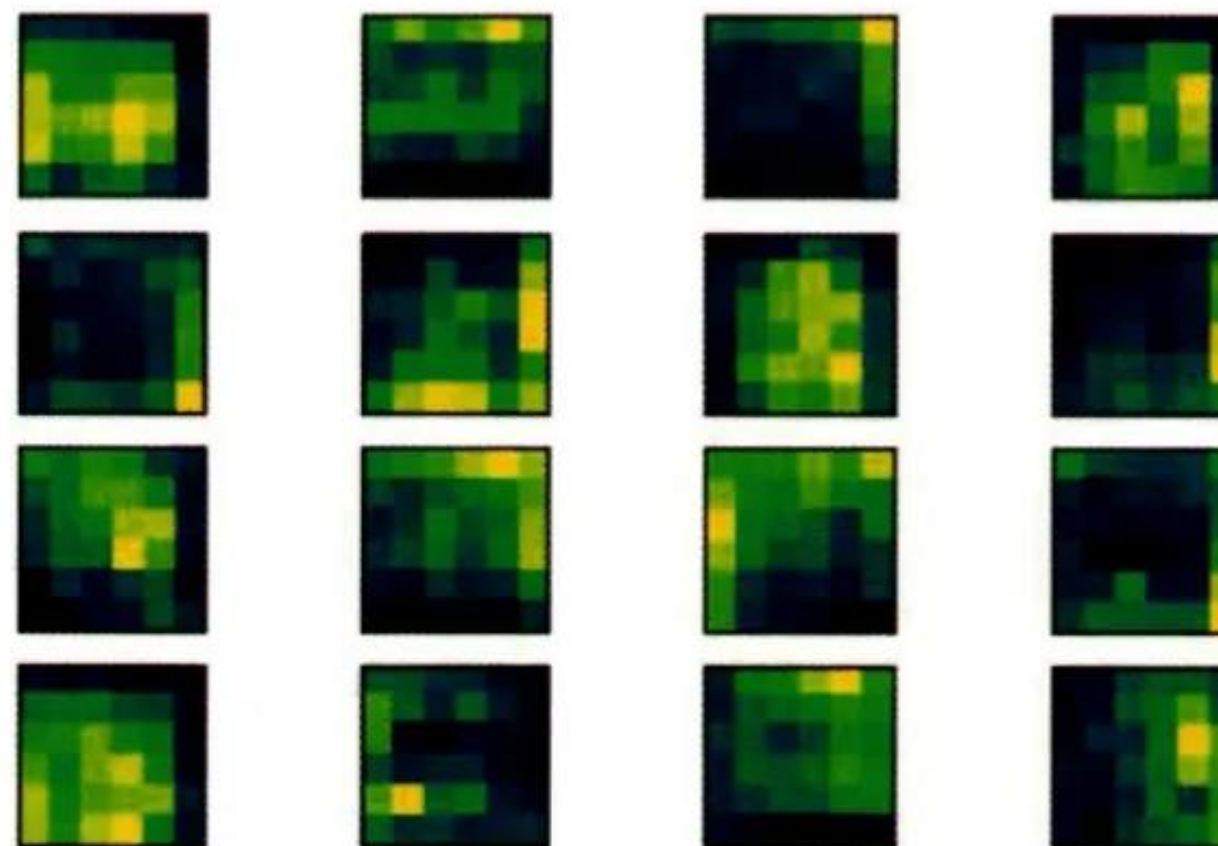
model(img)
activations = result.features
```

코드 5-40 특성 맵 확인

```
fig, axes = plt.subplots(4, 4)
fig = plt.figure(figsize=(12,8))

fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()
```

▼ 그림 5-49 40번째 계층에서 특성 맵

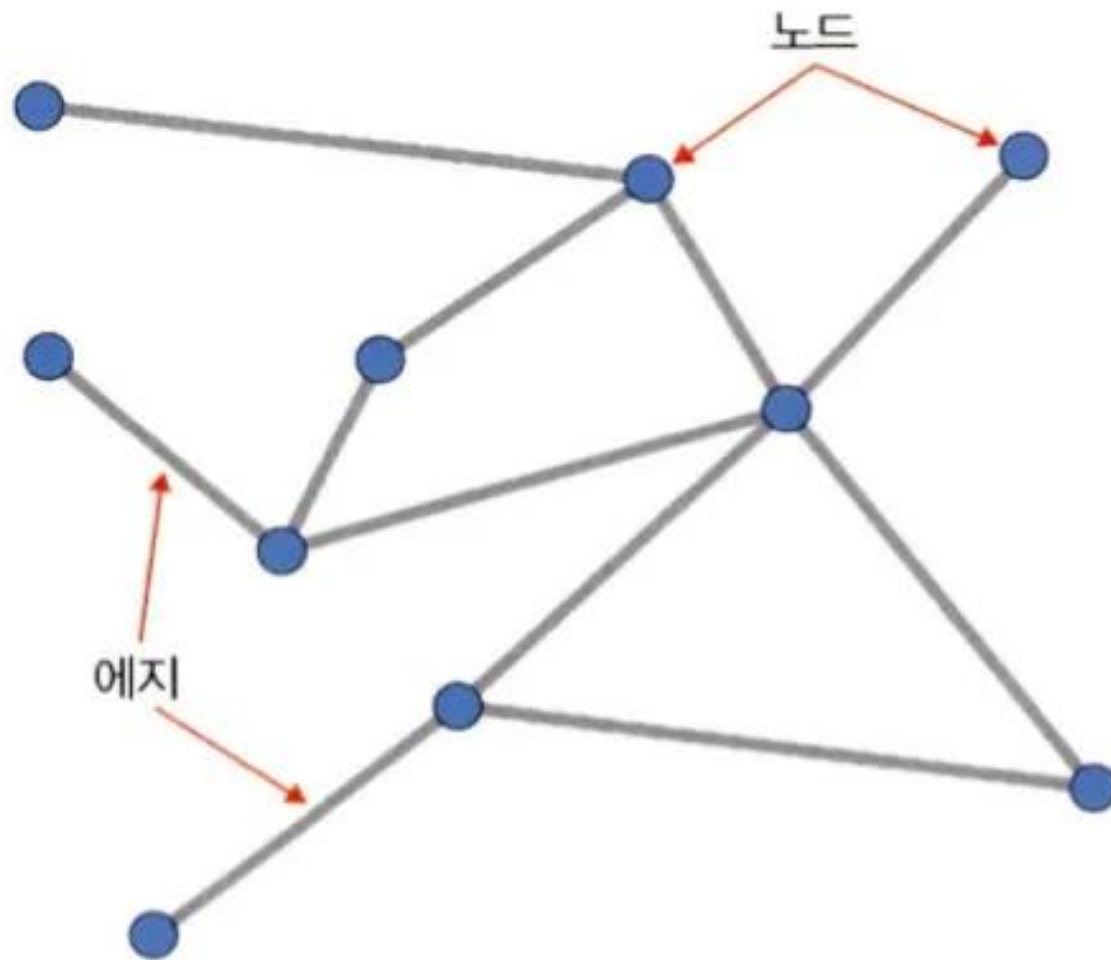


5.5 그래프 합성곱 네트워크



5.5.1 그래프란

▼ 그림 5-50 그래프



그래프: 방향성이 있거나 없는 에지로 연결된 노드의 집합

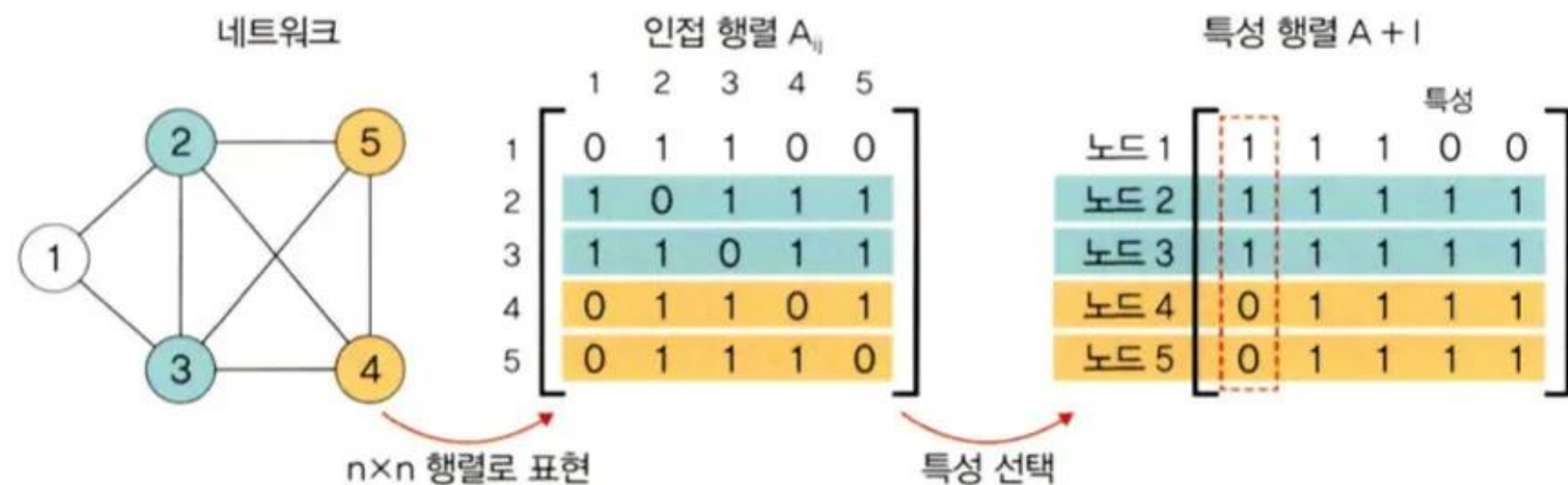
노드와 에지는 문제에 대한 지식/직관으로 구성

구성요소

- 노드(node, vertex): 파란색 원
- 에지(edge): 두 노드를 연결한 선

5.5.2 그래프 신경망

▼ 그림 5-51 특성 행렬⁵



그래프 신경망(GCN)

: 그래프 구조에서 사용하는 신경망

1단계 인접 행렬(adjacency matrix)

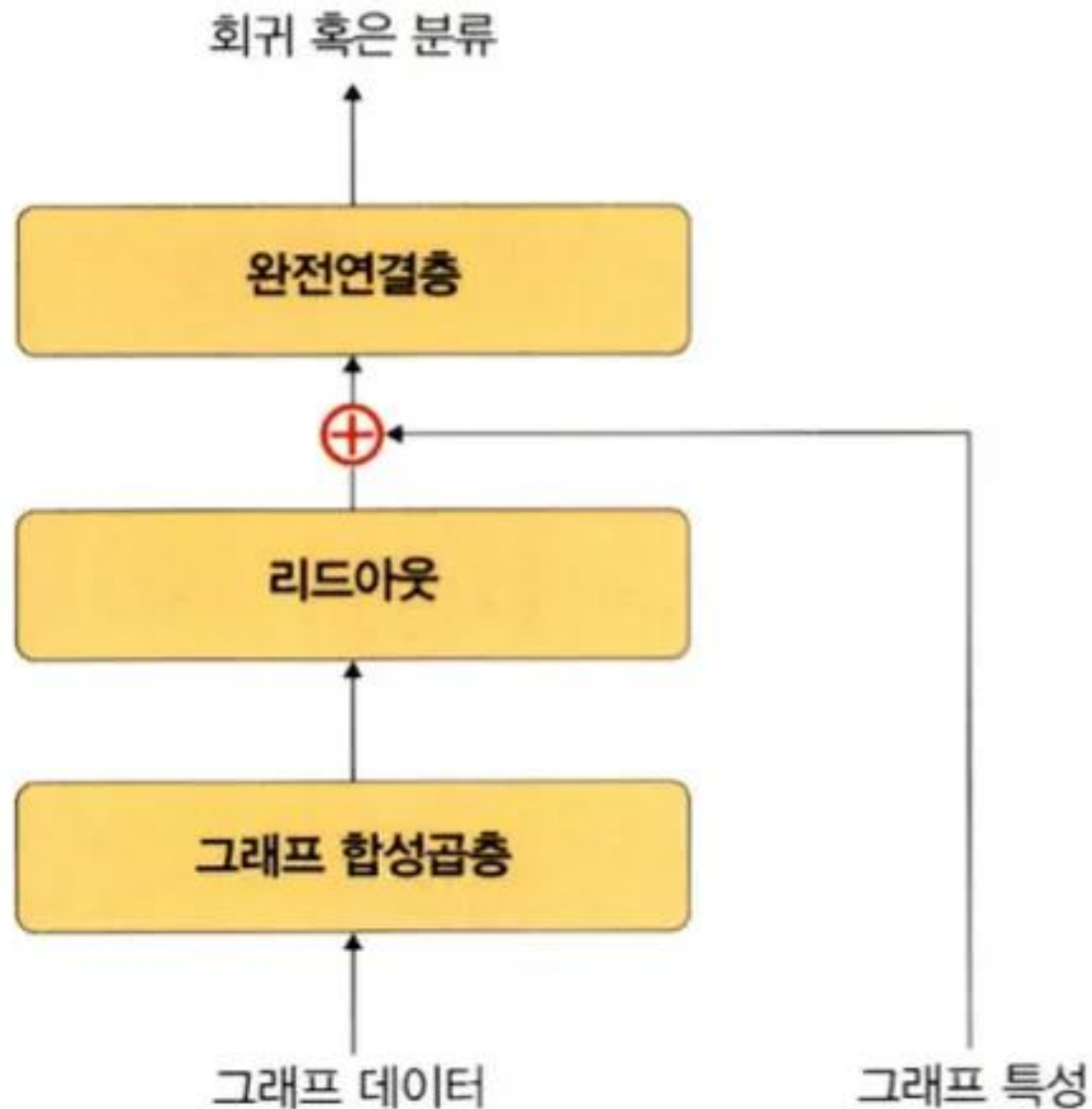
- 노드 n 개를 $n \times n$ 행렬로 표현
- 목적: 컴퓨터가 이해하기 쉽게 그래프를 행렬로 표현

2단계 특성 행렬(feature matrix)

- 인접 행렬만으로 특성 파악 어려워 단위 행렬 적용
- 각 입력 데이터에서 이용할 특성 선택
- 특성 행렬에서 각 행은 각 노드의 선택된 특성 값

5.5.3 그래프 합성곱 네트워크(GCN)

▼ 그림 5-52 그래프 합성곱 네트워크



GCN

- 이미지 합성곱을 그래프 데이터로 확장한 알고리즘
- 리드아웃: 특성 행렬을 하나의 벡터로 변환
- 전체 노드 특성 벡터의 평균으로 그래프 전체를 표현하는 벡터 생성
- 그래프 형태 데이터를 행렬형태로 변환
- -> 딥러닝 적용 가능
- 활용 예시
 - SNS 관계 네트워크
 - 학술 인용 네트워크
 - 3D Mesh

THANK YOU

